

Swarmbly AI

The barrier to serving artificial intelligence stops being capital and becomes participation

Sebastián A. Espinoza-Ulloa, Ph.D. · Independent researcher Whitepaper v2.0 (doi:10.5281/zenodo.23031305) · specification v0.2 · reference implementations, validation harness and measurements published AGPL-3.0-or-later (software) · CC BY 4.0 (text) · github.com/Sebastardito/Swarmbly-AI

The asymmetry

The knowledge to build artificial intelligence is public. Model weights, training recipes and inference engines are published openly and improve every month. **The capital to operate it is not.** Data centres consumed 415 TWh in 2024 — about 1.5 % of world electricity — with projections to 945 TWh by 2030, and that growth path runs through construction, which is available only to whoever can finance it.

So a technology whose knowledge belongs to everyone ends up controlled by whoever can afford the buildings. Not through a patent. Through a power contract.

Meanwhile the hardware already exists, switched on and idle. The flagship volunteer-computing platform today aggregates roughly **700,000 active devices, 4 million CPU cores, 560,000 GPUs and 93 PetaFLOPS** — from a community that has *shrunk* by 80 % over two decades. That number is a floor, drawn from a single declining niche, not a projection. At the level of an individual machine, an idle RTX 4090 is reported to serve language-model inference at **\$0.111-0.149 per million tokens**, at 62-78 % of H100 throughput for roughly half the cost.

The world's spare inference capacity is not a hypothesis. What has been missing is a protocol under which it can be used.

Why nobody has managed it yet

Every serious attempt so far has split the **model** — distributing transformer layers across machines so that intermediate activations cross the public internet on every generated token. That design runs head-first into a wall of physics: datacenter interconnect moves 900 GB/s, consumer upstream moves about 60 Mbps. **A ratio of roughly 120,000×**, and four to five orders of magnitude on latency.

The measured results match the prediction. Petals, the reference implementation of this approach, drops 31 % of its throughput to the network alone when moved from a lab link to a realistic one; a real geodistributed swarm of fourteen servers manages 0.83 steps per second.

That is not a bad implementation. It is the right answer to the wrong question.

The reframe

Swarmbly asks a different question: not *how do you run one large model across many machines*, but **how do you run many complete small models on one large problem**.

A small orchestrator on the user's own computer decomposes a request into semantic micro-tasks. Each is dispatched **once**, asynchronously, to a volunteer node running a complete small model. The returned fragments — *contigs*, in the genome-assembly vocabulary the design borrows deliberately — are verified, selected and spliced back together locally.

The network is crossed once per fragment per session, instead of once per layer per token. That single change moves the architecture from the side of the 120,000× wall where it loses, to the side where consumer hardware can participate at all. Splitting a model creates a chain, where every machine waits on the one before it. Splitting a problem creates a set, where they all work at once. That contrast is architectural: Swarmby's own throughput and latency have not been measured yet, and this page makes no speed claim on their behalf.

What has already been measured

Two things are now measured, and they point in different directions. Both are stated here.

Fragmenting has a cost, and the project's own abandonment test cannot yet rule on it. A go/no-go threshold was registered publicly *before any data existed*: if the loss never fell below 5 % in any task category, the architecture was to be abandoned. On the first held-out test, 16 prompts, the loss was **+2.30 %**, 95 % CI [**-2.05 %**, **+7.49 %**]. The criterion was written against the upper bound, which misses the threshold, so the verdict was **not met** — and the threshold was not moved. The cost was bimodal: 11 of 16 prompts lost nothing or less, and two prompts carried the whole mean. A second campaign, **473 runs** over five families of small models, measured the cost at an equal output budget: **+11.17 %**, 95 % CI [**+4.80**, **+17.34**]. That aggregate is still confounded with how much each arm writes, so the published harness **refuses** to call the criterion met or failed. It is reported as *not measured*.

Where the design says fragmenting should win, it does. On structured extraction — reading the records out of a long document and answering questions over all of them — a small model extracting the pieces, with code aggregating them, beats the same model answering alone by **+46.9 points**, 95 % CI [**+33.3**, **+59.4**]. A control that extracts the whole document in a single call separates the two mechanisms. At 80 rows the whole gain comes from taking the arithmetic away from the model, and splitting adds nothing. At 160 rows single-call extraction collapses and only splitting holds it up, by **+77.1 points**. A small node extracts reliably up to some task size, and fragmenting is what keeps each task below that size. That is the thesis of the design, measured directly for the first time. It is also narrow — 8 documents, one model family, two sizes — and it is stated as narrow.

An earlier version of this page reported a loss falling 24.1 % → 13.7 % as shared context rose, and said the prediction held. That table is withdrawn: the context axis had not actually moved, and the metric was not arm-neutral. The withdrawal is kept in full, with its arithmetic, in docs/RESULTS_V0_V3C.md.

What has not been proven — stated here, not buried

One published contribution did **not** survive. The architecture returns a *confidence map* — because independent model families answer the same micro-task, their agreement can be scored per unit, and a single centralized provider has nothing to align. The mechanism works. The claim that agreement predicts quality does not. The first measurement found **no relationship** ($r = -0.030$ over 597 units) against a weak judge that accepted 93 % of everything, so the honest verdict then was *unsupported, not refuted*. Three later runs graded against an answer key instead put the common odds ratio at **3.47**, then **0.26**, then **1.24** —

above, below and astride 1 on the same question. That is not a weak signal; it is no signal, measured three times. The confidence map has been **withdrawn**, not demoted, and no reliability benefit is claimed for it.

The measurements are also small: models of 2-4B parameters on local hardware, and task families chosen because they can be checked. A signal to act on, not a benchmark.

Four things this project does not claim. It is **not faster than a commercial API** for someone who already owns the hardware to run one — single-node speculative decoding beats any fragmentation scheme on latency, and Swarmbly's own latency has not been measured. It does **not offer unlimited context**, only a much higher limit that sits on the user's machine instead of in a vendor's price tier. It is **not encryption**: fragmentation raises the cost of reconstruction and nothing more, which is why genuinely sensitive work is routed to a closed circle or kept entirely local. And it has **not demonstrated an environmental benefit** — the argument is strong, the measurement is not yet made, and the project commits to publishing it whatever it shows.

This section exists because a project that hides its first negative result has not earned the first positive one.

Why now, and what exists today

Small models crossed the capability line that makes this possible only recently; the bandwidth gap that killed model-splitting is not closing. The opportunity is a timing one.

Published and public as of September 2026: whitepaper version 2 in English and Spanish, with 135 references; a complete wire specification; two reference implementations; a validation harness that refuses when its instrument cannot decide; a 473-run benchmark that anyone can re-execute; and every measurement in full, including the withdrawn ones. Everything is AGPL-3.0-or-later so that a hosted deployment cannot close it, with a dated prior-art record.

What it needs next is a team and the means to run it at scale. The next questions — where the extraction threshold sits for each class of node, and whether a context budget exists that satisfies coherence, privacy, verifiability and worker capability at once — need more than one person's hardware. And the likeliest way this fails is not an engineering fault; it is that nobody connects. Volunteer computing has been declining for twenty years, and the best protocol in the world is worth nothing to an empty network.

The knowledge is already public. The hardware is already built. What remains is the protocol, and it is now on the table where anyone can check it.

Full technical argument: docs/WHITEPAPER_V2_EN.md. Plain-language version: docs/DIVULGACION_EN.md. Current results in full: docs/RESULTS_2026-09-25_refbench_EN.md; the first held-out test: docs/RESULTS_TABLES_FINAL_CORRECTED.md. The withdrawn first measurements, kept with the arithmetic: docs/RESULTS_V0_V3C.md. Spanish version of this page: ONEPAGER_ES.md.